

Ejercicio 1

```
import java.util.Arrays;
import java.util.List;
public class Main {
    public static void main(String[] args) {
        List<Integer> numeros = Arrays.asList(
            1, 2, 3, 4, 5
        );
        numeros.stream()
            .forEach(System.out::println);
    }
}
```

Ejercicio 2

```
import java.util.Arrays;
import java.util.List;
import java.util.Set;
import java.util.stream.Collectors;
public class Main {
    public static void main(String[] args) {
        List<Integer> numeros = Arrays.asList(
            1, 2, 3, 4, 4, 5
        );
        Set<Integer> conjunto =
            numeros.stream()
                .collect(Collectors.toSet());
        System.out.println(conjunto);
    }
}
```

Ejercicio 3

```
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;
public class Main {
    public static void main(String[] args) {
        List<String> textos = Arrays.asList(
            "java",
            "python",
            "lambda",
            "streams"
        );
        List<String> mayusculas =
            new ArrayList<>();
        for (String texto : textos) {
            mayusculas.add(
                texto.toUpperCase()
            );
        }
        System.out.println(mayusculas);
    }
}
```

Ejercicio 4

```
import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;
public class Main {
    public static void main(String[] args) {
        List<String> textos = Arrays.asList(
```

```

        "java",
        "python",
        "lambda",
        "streams"
    );
    List<String> mayusculas =
        textos.stream()
            .map(String::toUpperCase)
            .collect(Collectors.toList());
    System.out.println(mayusculas);
}
}

```

Ejercicio 5

```

import java.util.Arrays;
import java.util.List;
public class Main {
    public static void main(String[] args) {
        List<String> textos = Arrays.asList(
            "java",
            "python",
            "lambda",
            "streams"
        );
        boolean existe =
            textos.stream()
                .anyMatch(
                    texto -> texto.contains("x")
                );
        System.out.println(existe);
    }
}

```

```
}  
}
```

Ejercicio 6

```
import java.util.Arrays;  
import java.util.List;  
public class Main {  
    public static void main(String[] args) {  
        List<String> textos = Arrays.asList(  
            "java",  
            "python",  
            "lambda",  
            "streams"  
        );  
        textos.stream()  
            .filter(  
                texto -> texto.contains("a")  
            )  
            .forEach(System.out::println);  
    }  
}
```

Ejercicio 7

```
import java.util.Arrays;  
import java.util.List;  
public class Main {  
    public static void main(String[] args) {  
        List<String> textos = Arrays.asList(  
            "java",  
            "python",  
            "lambda",
```

```

        "streams"
    );
    long cantidad =
        textos.stream()
            .filter(
                texto -> texto.contains("a")
            )
            .count();
    System.out.println(cantidad);
}
}

```

Ejercicio 8

```

import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;

public class Main {
    public static void main(String[] args) {
        List<String> palabras = Arrays.asList(
            "casa",
            "coche",
            "java",
            "camino",
            "perro",
            "conejo"
        );
        List<String> resultado =
            palabras.stream()
                .filter(

```

```
        palabra ->
            palabra.startsWith("c")
    )
    .map(String::toUpperCase)
    .sorted()
    .collect(Collectors.toList());
resultado.forEach(System.out::println);
}
}
```

Ejercicio 9

Clase Empleado

```
public class Empleado {
    private String nombre;
    private int edad;
    public Empleado(String nombre, int edad) {

        this.nombre = nombre;
        this.edad = edad;
    }
    public String getNombre() {
        return nombre;
    }
    public int getEdad() {
        return edad;
    }
}
```

Main

```
import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;
public class Main {
    public static void main(String[] args) {
        List<Empleado> empleados = Arrays.asList(
            new Empleado("Adrian", 20),
            new Empleado("Mario", 35),
            new Empleado("Lucia", 40),
            new Empleado("Carlos", 25)
        );
        List<String> nombres =
            empleados.stream()
                .map(Empleado::getNombre)
                .collect(Collectors.toList());
        System.out.println(nombres);
    }
}
```

Ejercicio 10

```
import java.util.Arrays;
import java.util.List;
public class Main {
    public static void main(String[] args) {
        List<Empleado> empleados = Arrays.asList(
            new Empleado("Adrian", 20),
            new Empleado("Mario", 35),
            new Empleado("Lucia", 40),
            new Empleado("Carlos", 25)
        );
    }
}
```

```

    );
    long cantidad =
        empleados.stream()
            .filter(
                empleado ->
                    empleado.getEdad() > 30
            )
            .count();
    System.out.println(cantidad);
}
}

```

Ejercicio 11

```

import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;
public class Main {
    public static void main(String[] args) {
        List<Empleado> empleados = Arrays.asList(
            new Empleado("Adrian", 20),
            new Empleado("Mario", 35),
            new Empleado("Lucia", 40),
            new Empleado("Carlos", 25)
        );
        List<String> nombres =
            empleados.stream()
                .filter(
                    empleado ->
                        empleado.getEdad() > 30
                )

```



```
        .map(Empleado::getNombre)
        .collect(Collectors.toList());

    System.out.println(nombres);
}
}
```

Ejercicio 12

Clase Empleado actualizada

```
public class Empleado {
    private String nombre;
    private int edad;
    private String departamento;

    public Empleado(
        String nombre,
        int edad,
        String departamento) {
        this.nombre = nombre;
        this.edad = edad;
        this.departamento = departamento;
    }

    public String getNombre() {
        return nombre;
    }

    public int getEdad() {
        return edad;
    }

    public String getDepartamento() {
        return departamento;
    }
}
```

Main

```
import java.util.Arrays;
import java.util.List;
public class Main {
    public static void main(String[] args) {
        List<Empleado> empleados = Arrays.asList(
            new Empleado(
                "Adrian",
                20,
                "Sistemas"
            ),
            new Empleado(
                "Mario",
                35,
                "Marketing"
            ),
            new Empleado(
                "Lucia",
                40,
                "Sistemas"
            ),
            new Empleado(
                "Carlos",
                25,
                "Sistemas"
            ),
            new Empleado(
                "Adrian",
                50,
```

```

        "Sistemas"
    )
};

empleados.stream()
    .filter(
        empleado ->
            empleado.getDepartamento()
                .equalsIgnoreCase(
                    "Sistemas"
                )
    )
    .map(Empleado::getNombre)
    .distinct()
    .sorted()
    .forEach(System.out::println);
}
}

```

Ejercicio 13

Explicación

El método `reduce` sirve para reducir todos los elementos de un stream a un único resultado.

En este ejemplo suma todos los números de la lista.

```

import java.util.Arrays;

import java.util.List;

public class Main {

    public static void main(String[] args) {

        List<Integer> numeros = Arrays.asList(
            1, 2, 3, 4, 5

```

```
);  
int suma =  
    numeros.stream()  
        .reduce(  
            0,  
            (a, b) -> a + b  
        );  
System.out.println(suma);  
}  
}
```